```
/*
 * COMPLETE GUARDIAN ROBOT - All Sensors + Line Following
 * Commands:
 *   F, B, L, R, S (Car Control in Manual Mode)
 *   M (Toggle Mode: Manual/Auto Line Following)
 *   Y (Voice Horn + Buzzer), X (Headlight ON), x (Headlight OFF)
 *
 * Features:
 * - Manual Mode: Full Bluetooth car control
 * - Auto Line Following Mode: Robot follows black line automatically
 * - Ultrasonic Sensor: Auto stop & buzzer when obstacle detected
 * - Gas Sensor (A0): Voice alert + LED blink + buzzer when gas detected
 * - Sound Sensor (A5): Voice alert + LED blink + buzzer when loud sound detected
 * - Voice Horn: ISD1820 + buzzer when Y pressed
 * - Headlights: On/Off control with X/x
 *
 * Priority: GAS > SOUND > ULTRASONIC > MODE OPERATION
 */

// ============================================
// PIN DEFINITIONS
// ============================================

// Motor Control (L298N)
#define ENA 3     // Left motor speed
#define IN1 4     // Left motor direction 1
#define IN2 5     // Left motor direction 2
#define IN3 6     // Right motor direction 1
```

```
#define IN4 7     // Right motor direction 2
#define ENB 8     // Right motor speed

// IR Sensors for Line Following (TCRT5000)
#define LEFT_SENSOR  11     // Left IR sensor on D11
#define RIGHT_SENSOR 12     // Right IR sensor on D12

// ISD1820 Voice Module and Headlights (LEDs)
#define ISD_TRIGGER A3   // ISD1820 voice trigger pin
#define HEADLIGHT1 A1    // Headlight LED 1
#define HEADLIGHT2 A2    // Headlight LED 2

// Ultrasonic Sensor (HC-SR04)
#define TRIG_PIN 9      // Ultrasonic trigger pin
#define ECHO_PIN 10     // Ultrasonic echo pin

// Gas Sensor (MQ-2)
#define GAS_SENSOR A0    // MQ-2 gas sensor analog output

// Sound Sensor (KY-037)
#define SOUND_SENSOR A5  // KY-037 sound sensor analog output (AO)

// Buzzer
#define BUZZER A4       // Buzzer on A4

// ==========================================
// CONSTANTS
// ==========================================
```

```
// Motor speeds
#define SPEED_NORMAL 200
#define SPEED_TURN 180

// Line Following Speeds (slower for better tracking)
#define LINE_SPEED_FORWARD 90
#define LINE_SPEED_TURN 70

// Ultrasonic
#define OBSTACLE_DISTANCE 30

// Gas Sensor
#define GAS_THRESHOLD 300
#define GAS_ALARM_DURATION 5000
#define GAS_BLINK_INTERVAL 200

// Sound Sensor
#define SOUND_THRESHOLD 100
#define SOUND_ALARM_DURATION 5000
#define SOUND_BLINK_INTERVAL 200

// ============================================
// GLOBAL VARIABLES
// ============================================

// Mode selection
bool lineFollowingMode = false;  // false = Manual Mode, true = Line Following Mode
```

```
// Ultrasonic variables
bool emergencyStopActive = false;
unsigned long lastObstacleBlink = 0;
bool obstacleBeepState = false;

// Gas sensor variables
bool gasDetected = false;
unsigned long gasAlarmStartTime = 0;
unsigned long gasLastBlink = 0;
bool gasBlinkState = false;

// Sound sensor variables
bool soundDetected = false;
unsigned long soundAlarmStartTime = 0;
unsigned long soundLastBlink = 0;
bool soundBlinkState = false;

// Headlight state
bool headlightsWereOn = false;

// Line following timing
unsigned long lineLastBlink = 0;
bool lineBlinkState = false;

// ==========================================
// SETUP
// ==========================================
```

```
void setup() {
  Serial.begin(9600);

  // Motor pins
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);

  // IR Sensor pins
  pinMode(LEFT_SENSOR, INPUT);
  pinMode(RIGHT_SENSOR, INPUT);

  // ISD1820 and headlight pins
  pinMode(ISD_TRIGGER, OUTPUT);
  pinMode(HEADLIGHT1, OUTPUT);
  pinMode(HEADLIGHT2, OUTPUT);

  // Ultrasonic sensor pins
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  // Gas and sound sensor pins
  pinMode(GAS_SENSOR, INPUT);
  pinMode(SOUND_SENSOR, INPUT);
```

```
  // Buzzer pin
  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  // Make sure all outputs start OFF
  digitalWrite(ISD_TRIGGER, LOW);
  digitalWrite(HEADLIGHT1, LOW);
  digitalWrite(HEADLIGHT2, LOW);

  // Stop motors
  stopMotors();

  // Startup sequence
  startupSequence();

  // Signal ready
  Serial.println("========================================");
  Serial.println("GUARDIAN ROBOT - FULLY INTEGRATED");
  Serial.println("========================================");
  Serial.println("Commands: F, B, L, R, S (Manual Mode)");
  Serial.println("        M (Toggle Manual/Line Following Mode)");
  Serial.println("        Y (Horn), X (Light ON), x (Light OFF)");
  Serial.println("========================================");
  Serial.println("Current Mode: MANUAL");
  Serial.println("========================================");
}
```

```
// ==========================================
// MAIN LOOP
// ==========================================

void loop() {
  // 1. Check GAS SENSOR (HIGHEST PRIORITY)
  checkGasSensor();
  if (gasDetected) {
    handleGasAlarm();
    return;
  }

  // 2. Check SOUND SENSOR (SECOND PRIORITY)
  checkSoundSensor();
  if (soundDetected) {
    handleSoundAlarm();
    return;
  }

  // 3. Check ultrasonic sensor for obstacles
  checkObstacle();

  // 4. Check Bluetooth commands
  if (Serial.available() > 0) {
    char command = Serial.read();
    processCommand(command);
  }
```

```
  // 5. If in Line Following Mode, run line follower
  if (lineFollowingMode && !emergencyStopActive) {
    runLineFollower();
  }

  delay(50);
}

// ==========================================
// COMMAND PROCESSING
// ==========================================

void processCommand(char command) {
  switch(command) {
    // Mode Toggle
    case 'M': case 'm':
      lineFollowingMode = !lineFollowingMode;
      stopMotors();
      if (lineFollowingMode) {
        Serial.println("MODE: LINE FOLLOWING AUTO");
        Serial.println("Place robot on black line track");
      } else {
        Serial.println("MODE: MANUAL CONTROL");
        Serial.println("Use F, B, L, R, S to control");
      }
      break;

    // Car Control (only in Manual Mode)
```

```
case 'F': case 'f':
  if (!lineFollowingMode && !emergencyStopActive) {
    moveForward();
    Serial.println("FORWARD");
  } else if (lineFollowingMode) {
    Serial.println("In Line Following Mode - Press M for Manual");
  } else {
    Serial.println("OBSTACLE DETECTED - CANNOT MOVE FORWARD");
  }
  break;

case 'B': case 'b':
  if (!lineFollowingMode) {
    moveBackward();
    Serial.println("BACKWARD");
  } else {
    Serial.println("In Line Following Mode - Press M for Manual");
  }
  break;

case 'L': case 'l':
  if (!lineFollowingMode) {
    turnLeft();
    Serial.println("LEFT");
  } else {
    Serial.println("In Line Following Mode - Press M for Manual");
  }
  break;
```

```
case 'R': case 'r':
  if (!lineFollowingMode) {
    turnRight();
    Serial.println("RIGHT");
  } else {
    Serial.println("In Line Following Mode - Press M for Manual");
  }
  break;

case 'S': case 's':
  stopMotors();
  emergencyStopActive = false;
  Serial.println("STOP");
  break;

// Voice Horn + Buzzer
case 'Y': case 'y':
  triggerVoiceAndBuzzer();
  Serial.println("HORN");
  break;

// Headlight Commands
case 'X':
  digitalWrite(HEADLIGHT1, HIGH);
  digitalWrite(HEADLIGHT2, HIGH);
  headlightsWereOn = true;
  Serial.println("HEADLIGHTS ON");
```

```
    break;

  case 'x':
    digitalWrite(HEADLIGHT1, LOW);
    digitalWrite(HEADLIGHT2, LOW);
    headlightsWereOn = false;
    Serial.println("HEADLIGHTS OFF");
    break;

  default:
    break;
 }
}

// ============================================
// LINE FOLLOWING MODE
// ============================================

void runLineFollower() {
 int left = digitalRead(LEFT_SENSOR);
 int right = digitalRead(RIGHT_SENSOR);

 // Both sensors on BLACK line -> GO FORWARD
 if (left == 1 && right == 1) {
  lineForward();
  digitalWrite(HEADLIGHT2, HIGH);  // Green LED
  digitalWrite(HEADLIGHT1, LOW);   // Red OFF
  noTone(BUZZER);
```

```
}

// Left on BLACK, Right on WHITE -> TURN LEFT
else if (left == 1 && right == 0) {
  lineTurnLeft();
  digitalWrite(HEADLIGHT1, HIGH);
  digitalWrite(HEADLIGHT2, LOW);
  tone(BUZZER, 800, 30);
}

// Right on BLACK, Left on WHITE -> TURN RIGHT
else if (left == 0 && right == 1) {
  lineTurnRight();
  digitalWrite(HEADLIGHT1, HIGH);
  digitalWrite(HEADLIGHT2, LOW);
  tone(BUZZER, 1000, 30);
}

// Both sensors on WHITE -> LINE LOST -> STOP
else {
  stopMotors();

  unsigned long currentTime = millis();
  if (currentTime - lineLastBlink > 300) {
    lineLastBlink = currentTime;
    lineBlinkState = !lineBlinkState;
    digitalWrite(HEADLIGHT1, lineBlinkState);
    digitalWrite(HEADLIGHT2, lineBlinkState);
```

```
    }

    static unsigned long lastBeep = 0;
    if (currentTime - lastBeep > 500) {
      tone(BUZZER, 600, 100);
      lastBeep = currentTime;
    }
  }
}

// ============================================
// LINE FOLLOWING MOTOR FUNCTIONS
// ============================================

void lineForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, LINE_SPEED_FORWARD);
  analogWrite(ENB, LINE_SPEED_FORWARD);
}

void lineTurnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
```

```
  analogWrite(ENA, 0);
  analogWrite(ENB, LINE_SPEED_TURN);
}

void lineTurnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, LINE_SPEED_TURN);
  analogWrite(ENB, 0);
}

// ============================================
// GAS SENSOR FUNCTIONS
// ============================================

void checkGasSensor() {
  int gasLevel = analogRead(GAS_SENSOR);

  static unsigned long lastGasPrint = 0;
  if (millis() - lastGasPrint > 3000) {
    Serial.print("Gas Level (A0): ");
    Serial.println(gasLevel);
    lastGasPrint = millis();
  }

  if (!gasDetected && gasLevel > GAS_THRESHOLD) {
```

```
    activateGasAlarm(gasLevel);
  }
}

void activateGasAlarm(int gasLevel) {
  gasDetected = true;
  gasAlarmStartTime = millis();
  stopMotors();

  Serial.println("========================================");
  Serial.print(" ⚠️ ⚠️ ⚠️  GAS LEAK DETECTED! ⚠️ ⚠️ ⚠️ ");
  Serial.print(" | Level: ");
  Serial.println(gasLevel);
  Serial.println("========================================");

  triggerVoiceMessage();

  digitalWrite(HEADLIGHT1, HIGH);
  digitalWrite(HEADLIGHT2, HIGH);
  delay(200);
  digitalWrite(HEADLIGHT1, LOW);
  digitalWrite(HEADLIGHT2, LOW);
}

void handleGasAlarm() {
  unsigned long currentTime = millis();

  if (currentTime - gasAlarmStartTime >= GAS_ALARM_DURATION) {
```

```
    gasDetected = false;
    digitalWrite(HEADLIGHT1, headlightsWereOn ? HIGH : LOW);
    digitalWrite(HEADLIGHT2, headlightsWereOn ? HIGH : LOW);
    digitalWrite(BUZZER, LOW);
    Serial.println("Gas alarm stopped - System normal");
    return;
  }

  if (currentTime - gasLastBlink >= GAS_BLINK_INTERVAL) {
    gasLastBlink = currentTime;
    gasBlinkState = !gasBlinkState;

    digitalWrite(HEADLIGHT1, gasBlinkState);
    digitalWrite(HEADLIGHT2, gasBlinkState);

    if (gasBlinkState) {
      digitalWrite(BUZZER, HIGH);
    } else {
      digitalWrite(BUZZER, LOW);
    }
  }
}

// ==========================================
// SOUND SENSOR FUNCTIONS
// ==========================================

void checkSoundSensor() {
```

```
  int soundLevel = analogRead(SOUND_SENSOR);

  static unsigned long lastSoundPrint = 0;
  if (millis() - lastSoundPrint > 3000) {
    Serial.print("Sound Level (A5): ");
    Serial.print(soundLevel);
    Serial.print(" | Threshold: ");
    Serial.println(SOUND_THRESHOLD);
    lastSoundPrint = millis();
  }

  if (!soundDetected && soundLevel > SOUND_THRESHOLD) {
    activateSoundAlarm(soundLevel);
  }
}

void activateSoundAlarm(int soundLevel) {
  soundDetected = true;
  soundAlarmStartTime = millis();
  stopMotors();

  Serial.println("========================================");
  Serial.print("🔊🔊🔊 LOUD SOUND DETECTED! 🔊🔊🔊 ");
  Serial.print(" | Level: ");
  Serial.println(soundLevel);
  Serial.println("========================================");

  triggerVoiceMessage();
```

```
  digitalWrite(HEADLIGHT1, HIGH);
  digitalWrite(HEADLIGHT2, HIGH);
  delay(200);
  digitalWrite(HEADLIGHT1, LOW);
  digitalWrite(HEADLIGHT2, LOW);
}

void handleSoundAlarm() {
  unsigned long currentTime = millis();

  if (currentTime - soundAlarmStartTime >= SOUND_ALARM_DURATION) {
    soundDetected = false;
    digitalWrite(HEADLIGHT1, headlightsWereOn ? HIGH : LOW);
    digitalWrite(HEADLIGHT2, headlightsWereOn ? HIGH : LOW);
    digitalWrite(BUZZER, LOW);
    Serial.println("Sound alarm stopped - System normal");
    return;
  }

  if (currentTime - soundLastBlink >= SOUND_BLINK_INTERVAL) {
    soundLastBlink = currentTime;
    soundBlinkState = !soundBlinkState;

    digitalWrite(HEADLIGHT1, soundBlinkState);
    digitalWrite(HEADLIGHT2, soundBlinkState);

    if (soundBlinkState) {
```

```
    digitalWrite(BUZZER, HIGH);
   } else {
    digitalWrite(BUZZER, LOW);
   }
  }
}

// ==========================================
// ULTRASONIC SENSOR FUNCTIONS
// ==========================================

int getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) return 999;

  int distance = duration * 0.034 / 2;
  return constrain(distance, 0, 400);
}

void checkObstacle() {
  int distance = getDistance();
```

```
if (distance > 0 && distance < OBSTACLE_DISTANCE) {
  if (!emergencyStopActive) {
    emergencyStopActive = true;
    stopMotors();
    Serial.print("EMERGENCY STOP! Obstacle at ");
    Serial.print(distance);
    Serial.println(" cm");

    digitalWrite(HEADLIGHT1, HIGH);
    digitalWrite(HEADLIGHT2, HIGH);
    delay(200);
    digitalWrite(HEADLIGHT1, headlightsWereOn ? HIGH : LOW);
    digitalWrite(HEADLIGHT2, headlightsWereOn ? HIGH : LOW);
  }

  unsigned long currentTime = millis();
  if (currentTime - lastObstacleBlink >= 200) {
    lastObstacleBlink = currentTime;
    obstacleBeepState = !obstacleBeepState;

    if (obstacleBeepState) {
      digitalWrite(BUZZER, HIGH);
      digitalWrite(HEADLIGHT1, HIGH);
      digitalWrite(HEADLIGHT2, HIGH);
    } else {
      digitalWrite(BUZZER, LOW);
      digitalWrite(HEADLIGHT1, headlightsWereOn ? HIGH : LOW);
```

```
      digitalWrite(HEADLIGHT2, headlightsWereOn ? HIGH : LOW);
    }
  }

 } else {
  if (emergencyStopActive) {
    emergencyStopActive = false;
    digitalWrite(BUZZER, LOW);
    digitalWrite(HEADLIGHT1, headlightsWereOn ? HIGH : LOW);
    digitalWrite(HEADLIGHT2, headlightsWereOn ? HIGH : LOW);
    Serial.println("Path clear - Emergency stop released");
  }
  obstacleBeepState = false;
 }

 static unsigned long lastPrint = 0;
 if (millis() - lastPrint > 2000) {
  if (distance < 100) {
    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");
  }
  lastPrint = millis();
 }
}

// ============================================
// VOICE FUNCTIONS
```

```
// ============================================

void triggerVoiceMessage() {
  digitalWrite(ISD_TRIGGER, HIGH);
  delay(150);
  digitalWrite(ISD_TRIGGER, LOW);
}

void triggerVoiceAndBuzzer() {
  digitalWrite(BUZZER, HIGH);
  digitalWrite(ISD_TRIGGER, HIGH);
  delay(150);
  digitalWrite(ISD_TRIGGER, LOW);
  delay(300);
  digitalWrite(BUZZER, LOW);
}

// ============================================
// MOTOR CONTROL FUNCTIONS (Manual Mode)
// ============================================

void moveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, SPEED_NORMAL);
  analogWrite(ENB, SPEED_NORMAL);
```

```
}

void moveBackward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, SPEED_NORMAL);
  analogWrite(ENB, SPEED_NORMAL);
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, SPEED_TURN);
  analogWrite(ENB, SPEED_TURN);
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, SPEED_TURN);
  analogWrite(ENB, SPEED_TURN);
}
```

```
void stopMotors() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

// ============================================
// STARTUP SEQUENCE
// ============================================

void startupSequence() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(HEADLIGHT1, HIGH);
    digitalWrite(HEADLIGHT2, HIGH);
    delay(200);
    digitalWrite(HEADLIGHT1, LOW);
    digitalWrite(HEADLIGHT2, LOW);
    delay(200);
  }

  digitalWrite(BUZZER, HIGH);
  delay(200);
  digitalWrite(BUZZER, LOW);
```

```
  triggerVoiceMessage();
  delay(1000);

  Serial.print("Initial Gas Level (A0): ");
  Serial.println(analogRead(GAS_SENSOR));
  Serial.print("Initial Sound Level (A5): ");
  Serial.println(analogRead(SOUND_SENSOR));
}

// ============================================
// END OF CODE
// ============================================
```